Republic of the Philippines
BATANGAS STATE UNIVERSITY
The National Engineering University
Alangilan Campus
Golden Country Homes, Alangilan Batangas City, Batangas, Philippines 4200
Tel Nos.: (+63 43) 425-0139 local 2222 / 2221
E-mail Address: cics.alangilan@g.batstate-u.edu.ph | Website Address: http://www.batstate-u.edu.ph

College of Informatics and Computing Sciences

PROJECT PROPOSAL

OOP 101 – OBJECT-ORIENTED PROGRAMMING
Computer Science Department
1st Semester, AY 2026-2027

I. TEAM INFORMATION

Team Name:	javarice		
Team Number:	14	**Section:**	CS-2102

TEAM MEMBERS			
No.	**SR-Code**	**Name**	**GitHub Username**
1	25-06472	Zamora Emmanuel Joshua P.	joshuazamora130-cpu
2	25-00261	Manacop, Venice Anne M.	created-by-the-creator
3	25-07236	Malabanan, Jay-z M.	25-07236-source

II. PROJECT INFORMATION

Project Title

OrgMate: Organization Membership Administration & Transaction Engine

Project Background

Many school organizations face difficulties in managing membership records and membership fee transactions. Member information and payment records are often scattered across spreadsheets, chats, and other platforms, making it difficult for officers to maintain accurate records, identify members who have paid their dues, and determine each member's current membership status. Students may also need to manually provide their information and payment details to organization officers through different channels, which can result in incomplete or disorganized records. These challenges affect both students submitting their membership fees and organization officers responsible for managing membership and financial records. Therefore, the project aims to develop a **centralized and computerized web-based membership management system** that allows students to submit their membership and payment information online, while enabling authorized officers to manage records, review payment submissions, verify receipts, and monitor membership fee status efficiently.

Project Objectives

General Objective:

To develop a Java-based Organization Membership Fee Management System that provides separate functions for students and organization officers, allowing students to submit membership and payment information online while enabling authorized officers to manage

member records, verify payment submissions, monitor payment status, and generate reports.

Specific Objectives:

1. To develop a **student registration and account module** that allows students to provide and manage their basic membership information.
2. To develop a **membership fee submission module** that allows students to submit payment details, including payment amount, payment method, reference number, and payment receipt.
3. To implement **payment method management** that supports payment methods such as GCash, Maya, Bank Transfer, and Cash.
4. To provide **payment verification functionality** that allows authorized officers to review submitted payment information and receipts and approve or reject payment submissions.
5. To provide **payment status monitoring** that allows students to view the status of their submitted payment and allows officers to monitor members who have paid, have pending submissions, or have not yet paid.
6. To maintain **centralized membership and payment records** that automatically organize student-submitted information and allow authorized officers to search, update, and manage records.
7. To generate **membership and payment reports**, including the total amount of verified membership fees collected.
8. To apply **Object-Oriented Programming (OOP) concepts**, including encapsulation, inheritance, polymorphism, abstraction, and exception handling, in developing the system.

Target Users

Target User	**How They Will Use the System**
Student/Member	Access the system through a web browser, provide or confirm their membership information, submit membership fee details, upload their payment receipt, and view the status of their payment submission.
Secretary	Register and update member information, manage membership records, and check members' membership status.
Treasurer	Record membership fee payments, input member's payment method, update payment status, and monitor the total amount of collected fees.
Auditor	Verify payment records, review membership fee transactions, and generate or check payment reports to ensure that records are accurate.
President	Monitor the organization's overall membership status and membership fee collection, review summary reports, and oversee the accuracy and completeness of membership records.

Adviser	Review membership and payment summaries, monitor the organization's membership administration, and provide guidance or approval when necessary.

Proposed Features

Feature	Description
Student/Member Registration	Allows students to access the system and provide or confirm their membership information using their student credentials or SR-Code.
Student Membership Profile	Displays the student's registered information, such as name, SR-Code, program, and block.
Membership Fee Submission	Allows students to submit their membership fee information, including the amount paid, payment method, reference number, and payment date.
Membership Fee Management	Allows officers to record membership fee payments, including the amount and payment date.
Payment Method Selection	Allows officers to record the payment method used by members, such as GCash, Maya, Bank Transfer, or Cash.
Receipt Upload	Allows students to upload a digital copy or image of their payment receipt as proof of payment.
Payment Status Monitoring	Allows students and authorized officers to view the current status of a payment, such as Pending, Verified, or Rejected.
Payment Verification	Allows authorized officers to review submitted payment information and receipts and verify or reject payment submissions.
Centralized Membership Records	Stores member information and payment records in one centralized system for easier management and access.
Search and Record Management	Allows authorized officers to search, view, update, and manage membership and payment records.
Membership and Payment Reports	Generates reports containing membership information and payment transactions for monitoring and documentation.
Total Fee Collection Monitoring	Displays the total amount of verified membership fees collected by the organization.

III. JAVA AND OOP CONCEPTS

OOP Concept	How It Will Be Used
Classes and Objects	Classes such as `Student`, `RegularStudent`, `Officer`, `Payment`, `GCashPayment`, `MayaPayment`, `BankPayment`, `CashPayment`, and `MembershipManager` will represent the system's entities. Objects will be created from these classes to store and manage student, officer, and payment information.
Methods	Methods such as `addMember()`, `searchMember()`, `recordPayment()`, `processPayment()`, `updatePaymentStatus()`, `deleteRecord()`, `displayInfo()`, `getTotalCollected()`, and `generateReport()` will organize the system's functions.
Encapsulation	Student and payment information will be stored in `private` fields and accessed or modified through getter and setter methods to protect data and control access.
Inheritance	`RegularStudent` and `Officer` will inherit common attributes and behaviors from the `Student` class. Likewise, `GCashPayment`, `MayaPayment`, `BankPayment`, and `CashPayment` will inherit common attributes and behaviors from the `Payment` class. .
Polymorphism	The `Payment` class will define `processPayment()`, while `GCashPayment`, `MayaPayment`, `BankPayment`, and `CashPayment` will override the method to handle the recording and verification of each payment method.
Abstraction	`Payment` will be an abstract class that defines common payment information and the required `processPayment()` behavior while hiding the

	specific implementation details of each payment method.
Exception Handling	`try-catch` and appropriate exception handling will be used to handle invalid inputs, duplicate student records, invalid Student IDs, negative payment amounts, invalid payment methods, and missing required information.
Arrays/Data Handling	`ArrayList<Student>` and `ArrayList<Payment>` will be used to store and manage multiple student and payment records. The lists will support searching, updating, deleting, displaying records, checking duplicates, and calculating total verified fees.

IV. PROPOSED CLASSES

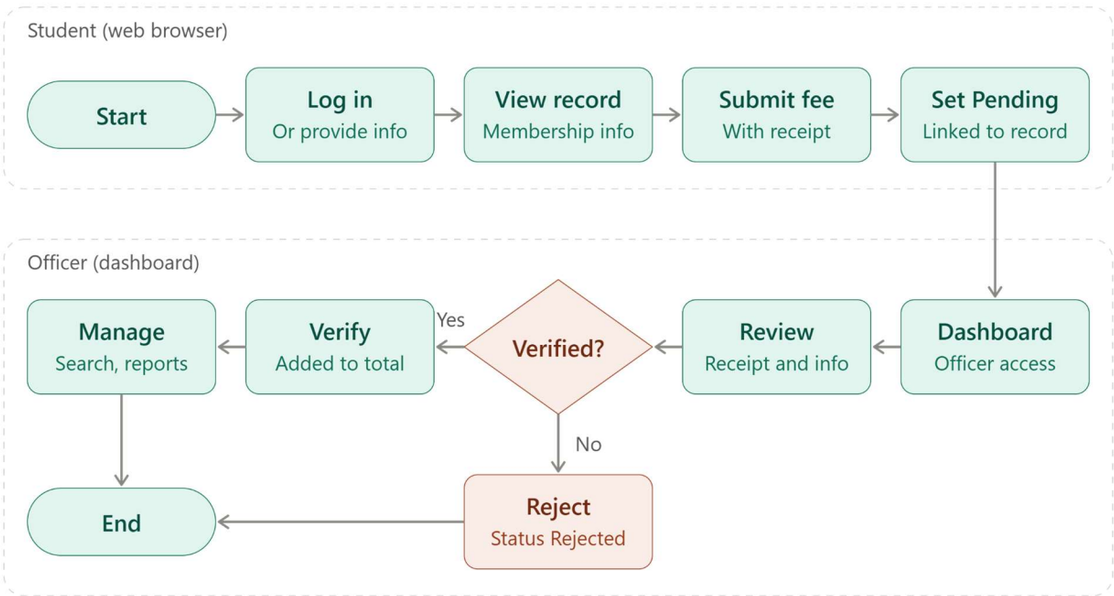
Class	Purpose	Main Attributes	Main Methods
Student	Stores shared student information. The officer adds access privileges.	studentName, studentID, block, department, program (Officer: position)	getters/setters, displayInfo()
Regular student	Represents a member who logs in, views their membership, and submits fee payments. Inherits from Student.	membershipStatus (Not Paid, Pending, Paid)	viewMembership(), submitPayment(), viewPaymentStatus(), displayInfo() (overridden)
Officer (parent)	Parent of all officer roles. Holds the access shared by every officer, but does not define what each role does. Inherits from Student.	officerID, position	getPosition(), searchRecords(), performDuty() (abstract), displayInfo() (overridden)
Payment	Defines what every payment must do, not how.	amount, paymentDate, status, reference No, status	processPayment(), getters/setters

College of Informatics and Computing Sciences

President	Oversees the organization and approves final reports.	none beyond inherited	approveReport(), viewOverallSummary(), performDuty() (overridden)
Secretary	Manages membership records only	none beyond inherited	addMember(), updateMember(), deleteRecord(), generateMembershipReport(), performDuty() (overridden
Treasurer	The only role that handles money. Reviews receipts and verifies or rejects payments.	none beyond inherited	reviewReceipt(), verifyPayment(), rejectPayment(), getTotalCollected(), generatePaymentReport(), performDuty() (overridden)
Auditor	Checks that payment records are accurate. Cannot verify or change payments.	none beyond inherited	auditTransactions(), flagDiscrepancy(), generateAuditReport(), performDuty() (overridden)
Adviser	Read-only oversight. Monitors records and payments but cannot change them.	none beyond inherited	viewRecords(), viewPaymentLog(), addRemarks(), performDuty() (overridden)
GCashPayment, MayaPayment, BankPayment, CashPayment	Each payment method overrides the parent's behavior.	Method-specific data, such as receiptNo for Cash	processPayment() (overridden)
Membership Manager	Manages all records using ArrayLists.	ArrayList<Student>, ArrayList<Payment>	addMember(), searchMember(), recordPayment(), updatePaymentStatus(), deleteRecord(), getTotalCollected(), generateReport()

V. PROGRAM FLOW

The system provides different functions depending on the user's role. Students access the web application through a browser and log in or provide their required student information. After accessing their account, students can view their membership information and submit their membership fee details, including the amount paid, payment method, reference number, and payment receipt. The system automatically associates the submitted payment with the student's membership record and assigns a Pending status. Authorized officers can access the officer dashboard to view submitted payments, review the uploaded receipts and payment information, and verify or reject the submissions. Once a payment is verified, the system updates the student's payment status and automatically includes the verified amount in the total membership fee collection. Officers can also search and manage records and generate membership and payment reports.



VI. TEAM RESPONSIBILITIES

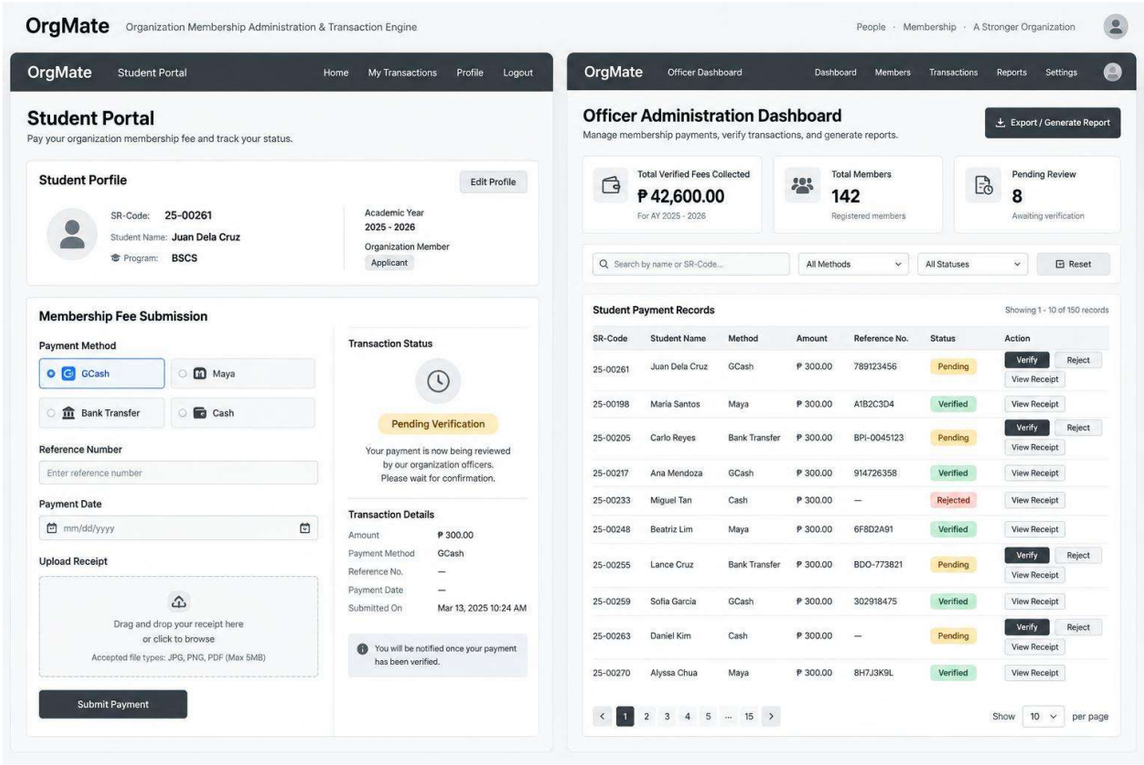
Member	Assigned Tasks
Zamora Emmanuel Joshua P.	**Testing & Development Lead, Program Developer** - Develop system modules, assist with implementation, and conduct system testing and debugging.
Manacop, Venice Anne M.	**Project & System/Design Lead, Lead Programmer** - Lead system planning and design, coordinate project development, and implement major system components.

Malabanan, Jay-z M.	**Testing & Development Member, Program Developer** - Assist in system development, implement program functions and modules, conduct testing and debugging, and ensure proper integration of system components.

VII. DEVELOPMENT SCHEDULE

Week	Activity	Expected Output
Week 1	Finalization of group information, team name, project idea, project background, objectives, and target users	Completed and finalized initial project proposal details and submitted project idea
Week 2	Drafting and completing the required sections of the Project Proposal	Initial draft of the Project Proposal with the required sections completed and prepared for finalization
Week 3-4	Java coding and development of the system's core classes, data handling, membership functions, and payment submission/verification functions	Working backend/core system with basic membership and payment functionality
Week 5	Implementation and integration of OOP concepts	OOP concepts fully integrated into the system
Week 6	Development of the web UI, including separate student and officer interfaces	Functional student and officer interfaces
Week 7	System testing, debugging, error fixing, and code cleanup	Tested and refined web application
Week 8	Finalization, documentation, preparation of presentation materials, and project presentation	Final working system, completed documentation, and project presentation

VIII. EXPECTED OUTPUT



IX. PROJECT LIMITATIONS

The system is limited to organization membership and membership fee management. Students can access the system through a web browser to submit their membership information and payment details, while authorized officers can review and verify submitted payments and manage records. The system records payments made through GCash, Maya, Bank Transfer, and Cash but does not directly integrate with or process transactions through external payment platforms. Payment verification is performed manually by authorized officers based on the submitted payment information and receipt. The system does not cover other financial transactions such as event fees, donations, or organizational expenses. Data management is limited to the storage and capabilities implemented by the system and does not include integration with the school's official student information system. The system does not provide automated payment verification, advanced analytics, or direct integration with external financial platforms.

X. GITHUB REPOSITORY

Repository Name: OrgMate
Link to Repository: https://github.com/joshuazamora130-cpu/OrgMate.git